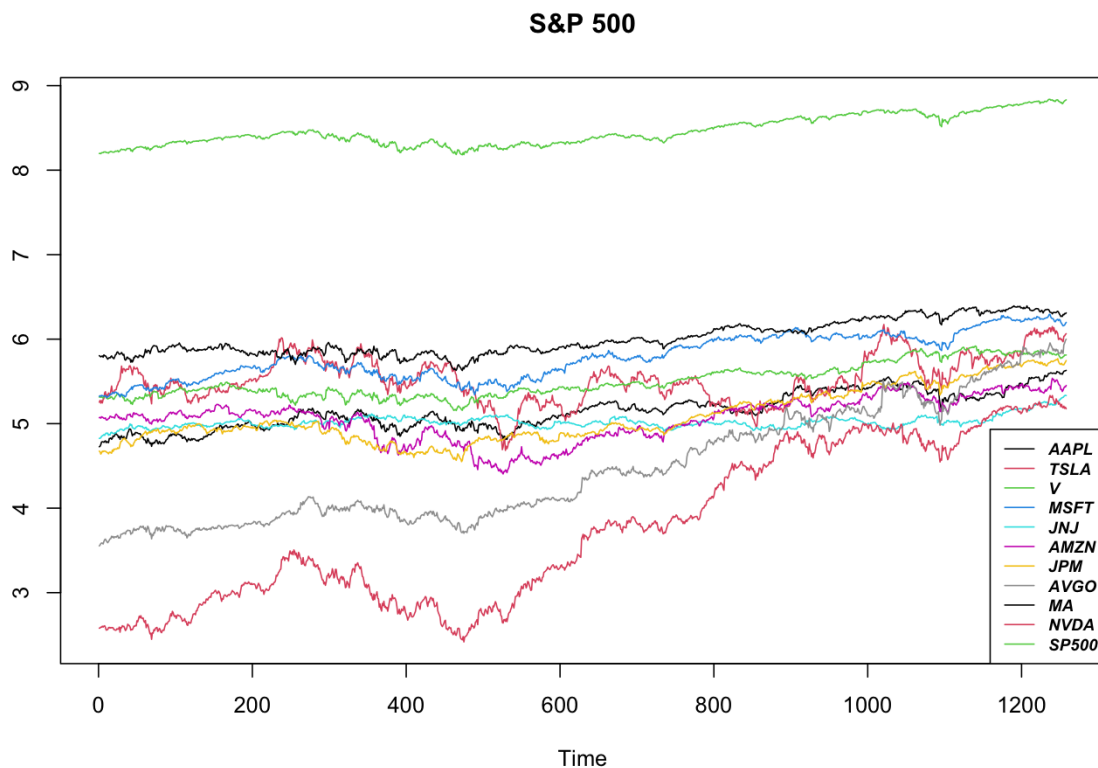


ST326 Coursework

Candidate number: 69729

Q1

The chosen stocks are AAPL, TSLA, V, MSFT, JNJ, AMZN, JPM, AVGO, MA, NVDA along with the S&P 500 (named SP500 in the code) we have our set of 11 stocks. The date range for the data is the last 5 working years from 2020-11-25 to 2025-11-30. The [quantmod](#) package has been used to attain the adjusted closing prices. The data is from the top 50 constituents of the S&P 500 so there were no issues with missing values. The other packages used are [tidyverse](#) and [reshape](#) for formatting and plotting. The log prices were then manually merged and plotted below:



We can see that dips and highs are similar for the constituents stocks while the index itself is not prone to large dips. This trend becomes important in the scope of **Q5** as it is a sign of multicollinearity which **Q5** aims to rectify.

Q2

To split the data into training, validation and test sets I utilized the function provided in the Moodle with some alterations. The returns were calculated using the `diff()` function on the adjusted prices, and then the return data was merged into one `.xts` type object which was later coerced into a matrix for our data preparation. The function `pred.footsie.prep()` (taken from the Moodle) was modified to accommodate for 10 stocks instead of 9 indices and the data was changed to `r.mtrx`.

To estimate daily volatilities using exponential smoothing our first step is to find the relevant lambdas. Note the assumed model for each series is an IGARCH(1,1) process. We initialise $\sigma_1^2 = x_1^2$. Following the discussion from exercise 5 we equivalently maximise the conditional density:

$$\prod_{n=2}^T \frac{1}{\sqrt{2\pi\sigma_t^2}} \exp\left(-\frac{x_t^2}{2\sigma_t^2}\right), \quad \sigma_t^2 = (1-\lambda) \sum_{j=0}^{t-2} \lambda^j x_{t-j-1}^2 + \lambda^{t-1} x_1^2.$$

The log-likelihood function (in the code named `log.lik.fcn()`) is

$$\sum_{n=2}^T \left[\log\left(\frac{1}{\sqrt{2\pi\sigma_t^2}}\right) - \frac{x_t^2}{2\sigma_t^2} \right]$$

The function `mle.train()`¹ finds lambda using `optim()` and returns a list with lambda for each return series. The function `vol.exp.sm()` uses the MLEs provided by `mle.train()` to find the daily volatilities and normalises data. Finally, the function `norm.data()` was initialised to normalise all the data provided by `pred.r.prep()`.

¹Note we find lambda over the training set and use that as our lambda similar to the lecture notes.

Q3

Prediction Algorithm

- I. We start at a certain time $t = t_0$ with window length D having normalised the data using the relevant functions in **Q2**. We write our model as

$$\mathbf{Y} = \mathbf{Z}\boldsymbol{\alpha} + \mathbf{e},$$

Where $\mathbf{Y} = (Y_{t-D+1}, \dots, Y_t)^T$, $\mathbf{Z} = (\mathbf{1}_D, \mathbf{Z}^1, \dots, \mathbf{Z}^{q+10})$ with $\mathbf{Z} = (Z_{t-D}^k, \dots, Z_{t-1}^k)^T$, $\boldsymbol{\alpha} = (\alpha_0, \dots, \alpha_{q+10})^T$ and $\mathbf{e} = (e_{t-D+1}, \dots, e_t)^T$. $\mathbf{1}_D$ is the column vector of ones of dimension D , and we estimate alpha using Least Squares Estimation:

$$\hat{\boldsymbol{\alpha}} = (\mathbf{Z}^T \mathbf{Z})^{-1} \mathbf{Z}^T \mathbf{Y}$$

- II. So our predicted return at time

$$\hat{Y}_{t+1} = \hat{\boldsymbol{\alpha}}^T \mathbf{z}_t$$

where $\mathbf{z}_t = (1, Z_t^1, \dots, Z_t^{q+10})$

- III. We have a different investment strategy compared to the lecture notes, we first define a new function and then calculate returns R_{t+1} .

$$I(\hat{Y}_{t+1}) = \begin{cases} 1 & \hat{Y}_{t+1} > 0 \\ -1 & \hat{Y}_{t+1} < 0 \end{cases}$$

$$R_{t+1} = I(\hat{Y}_{t+1})Y_{t+1}$$

- IV. We return to step I., but with t replaced by $t + 1$. Stop when t reaches the end of the training set.

- V. We calculate the **Sharpe ratios** using

$$(\text{Annualised}) \text{ Sharpe ratio} = \sqrt{250} \frac{\text{Average daily return}}{\text{SD of daily returns}}$$

We repeat steps I. to IV. for a grid of values of D .

Q4

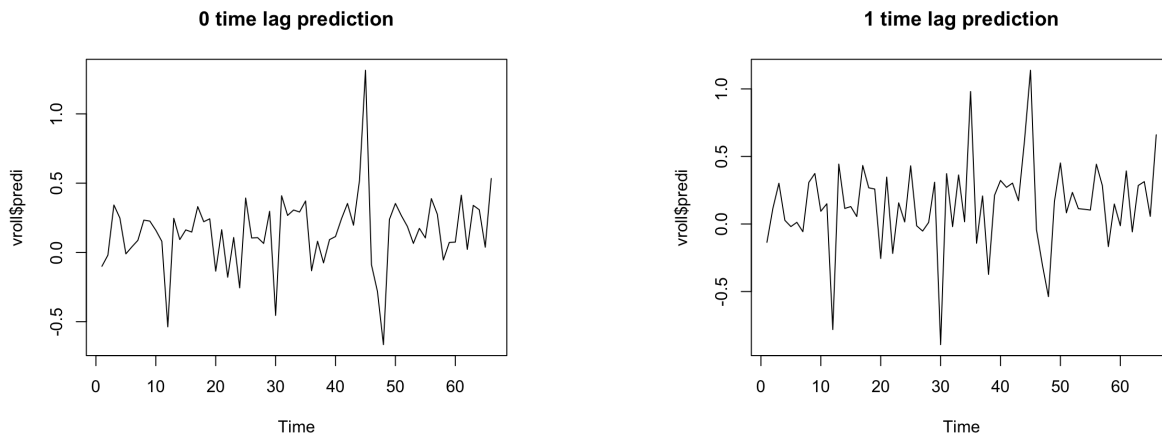
The functions² `sharpe.curves()`, `thresh.reg()` and `rolling.thresh.reg()` were created to run the algorithm above. The `sharpe.curves()` function computes the Sharpe ratios for the rolling windows for training, validation and test sets. We have used a rolling window of $D = 50$ to 250 adding 20 each time. The Sharpe ratios for $q = 0$ and $q = 1$ are called by `sc0` and `sc1` respectively.

```
> sc0$train.curve
[1] -0.1731737  0.3894907 -0.5614358 -0.1355825  0.5352758  0.5455107  0.4083102
↪ -0.4859850 -0.2669667  0.5146608 0.2133667
> sc0$valid.curve
[1] -0.7961328  0.7957651  2.9704650  5.1672774  3.3625409  2.8853777  3.5700361
↪ 3.2768424  3.0079619  5.3917946  5.1734213
> sc0$test.curve
[1]  2.8794471  0.6869675 -0.8417383  0.4210184 -0.6999320 -0.3367436 -0.1069790
↪ -1.5387527 -1.1937269 -0.5474109 -0.5267639
```

```
> sc1$train.curve
[1] -0.09450712  0.30872740 -0.31423544 -0.24441217  1.07962940  0.60111400
↪ 0.16154768 -0.34164445  0.27804637 0.38954837  0.23296821
> sc1$valid.curve
[1] -1.313469  3.367742  1.290821  3.822369  4.351837  5.489923  4.178909  2.694727
↪ 3.612576  2.973056  5.049770
> sc1$test.curve
[1]  2.63086668 -0.13963347 -0.50491318  0.01098378 -0.67846841 -1.06207328
↪ -0.10829266 -1.98130209 -2.32560115 -1.68608908 -2.11784687
```

For $q = 0$, note that the Sharpe ratios are positive for only the validation set, suggesting the ordinary least square is not the most appropriate model to use, this idea is strengthened when looking at the plots (below) which show extremely risky positions in the market such as investing all of your money into the index which is not sound investment strategy. For $q = 1$ the story is similar as the Sharpe ratios are acceptable for some of the training set and validation set. However the prediction of OLS is still not good enough for real-life investment strategy. In aggregate this tells us that we need a better model.

²take from the moodle



Q5

To conduct a principal component regression I created a function `fctr.mdl.train()` dependent on window length and factors. The function first separates the data into training, validation and test sets by calling `pred.r.prep(1)` then normalises the data using `norm.data()`. After this within each set, in each window we perform PCR by subtracting the mean for the data within the window. Now, having standardised the data by subtracting the mean, we call on the function `prcomp()` and conduct the code³:

```
#xxx is our standardised data (de-meanned and normalised)
train.pca <- prcomp(xxx)
f2<-t(train.pca$rotation[,1])%*t(train.pca$x)
```

Following from the discussion in the lectures `f2` becomes the covariates on which the linear model is used. A new function `sharpe.curves.fctr()` dependent on window length and factors was used to find the Sharpe ratios. We have used a rolling window of $D = 50$ to 250 adding 20 each time. The Sharpe ratios for 1 factor and 2 factors are called by `scf1` and `scf2` respectively.

```
> scf1$train.curve
[1] -0.739387203 -0.919849314 -0.364650239  0.050545659  0.087641088  0.233693687
↪ -0.229759320 -0.679411854  0.044017127 -0.004664653  .077621381
> scf1$valid.curve
```

³The code is slightly different for 2 factors see appendix for details

```

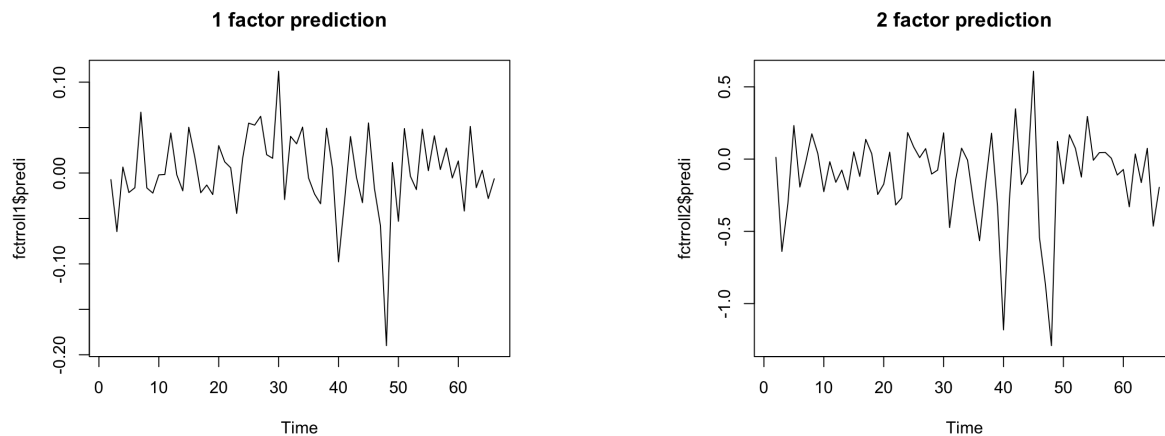
[1] -3.5742566 -2.3849609 -3.1584906 -1.5212635 -1.4669264 -1.3158870  0.4792984
↪ -3.7511987 -1.1150365 -2.7541971 -1.1241751
> scf1$test.curve
[1] -1.5075002 -1.8103211 -1.8103211 -0.9062674 -0.9062674 -1.1992948 -1.5075002
↪ -1.5075002 -1.5021882 -2.4792536 -0.1062624

```

```

> scf2$train.curve
[1]  0.08205315  0.10468574 -0.51535217 -0.13591428 -0.31231085  0.09323230
↪ -0.84832470  0.26302018 -0.80956161 -0.35640407  0.2957234
> scf2$valid.curve
[1] -0.7464877  1.0390573 -0.4280399  2.5017733  0.6357039 -1.5231966 -0.8215945
↪ -1.9690268 -0.7714236 -1.3528861  1.2305910
> scf2$test.curve
[1] -1.9075350 -3.7698852  1.8657730  1.3395244  0.4194532 -0.5583602 -1.3220467
↪ -1.7376589  1.8460125  2.2607119  .3794921

```



This method may be better than least squares as the one factor predictions are much more reasonable than extremely risky predictions given by OLS. The reason for this must be the reduction of multicollinearity which seemed to have heavily affected the results as mentioned in **Q1**. However the two factor prediction still seems to give unreasonable positions so it is not as clear that this model is clearly better. Another issue with this model is that it doesn't suit our investment strategy, as the Sharpe ratios of this model are much higher and mostly negative. All this means that given our different investment strategy it might be better to use OLS.

Appendix

I have divided the code by each question; however, the code of the first few questions should be run to ensure that all the relevant functions are present for the later questions. Generative AI has been used to spellcheck and debug some functions.

A Question 1 Code

```
install.packages(c("tidyverse","quantmod"))
install.packages("reshape2")
library(eodhdr2)
library(quantmod)
library(readr)
library(dplyr)
stock.names<-c( 'AAPL', 'TSLA', 'V', 'MSFT', 'JNJ', 'AMZN', 'JPM', 'AVGO', 'MA', 'NVDA',"SP500")

g<-c("^GSPC", 'AAPL', 'TSLA', 'V', 'MSFT', 'JNJ', 'AMZN', 'JPM', 'AVGO', 'MA', 'NVDA')

getSymbols(g,from="2020-11-25",to="2025-11-30")

log_prices<-merge(log(AAPL$AAPL.Adjusted), log(TSLA$TSLA.Adjusted),
                  log(V$V.Adjusted), log(MSFT$MSFT.Adjusted),
                  log(JNJ$JNJ.Adjusted), log(AMZN$AMZN.Adjusted),
                  log(JPM$JPM.Adjusted), log(AVGO$AVGO.Adjusted),
                  log(MA$MA.Adjusted), log(NVDA$NVDA.Adjusted),
                  log(GSPC$GSPC.Adjusted))

ts.plot(log_prices, col=1:11,main="S&P 500")

legend("bottomright",
      legend = stock.names,
      col = 1:11, text.font=4, cex=.7, lty=1,)
```

B Question 2 Code

The `pred.r.prep()` function is outlined in the code below

```
stock.names.2<-c( "SP500", 'AAPL', 'TSLA', 'V', 'MSFT', 'JNJ', 'AMZN', 'JPM', 'AVGO', 'MA', 'NVDA')
r<-merge(diff(GSPC$GSPC.Adjusted),diff(AAPL$AAPL.Adjusted), diff(TSLA$TSLA.Adjusted), diff(V$V.Adjusted),
diff(MSFT$MSFT.Adjusted),diff(JNJ$JNJ.Adjusted),
diff(AMZN$AMZN.Adjusted),diff(JPM$JPM.Adjusted),diff(AVGO$AVGO.Adjusted),diff(MA$MA.Adjusted),
diff(NVDA$NVDA.Adjusted))

r=r[-1]
colnames(r)<-stock.names.2
dim(r)

r.mtx = data.matrix(as.data.frame(r))
r.mtx <- t(r.mtx) #this transpose is important as our function pred.r.prepare will not work
```

```

      #if the data's dimension is incorrect we require stock names as rows
r.mtx

pred.r.prepare <- function(max.lag = 5, split = c(50, 25), mask = rep(1, 10)) {

  # this function prepares the data for the prediction exercise
  # and splits them into a train, validation and test sets
  # max.lag - the maximum lag to include in the prediction
  # split - how much of the data (in percentage terms)
  # to include in the training and validation sets, respectively
  # mask - which other stocks to include (1 for yes, 0 for no)
  r.mtx <- as.matrix(r.mtx)

  d <- dim(r.mtx)

  start.index <- max(3, max.lag + 1)

  y <- matrix(0, d[2] - start.index + 1, 1)

  x <- matrix(0, d[2] - start.index + 1, d[1] - 1 + max.lag)

  y[,1] <- r.mtx[1,start.index:d[2]]

  for (i in 1:max.lag) {
    x[,i] <- r.mtx[1,(start.index-i):(d[2]-i)]
  }

  shift.indices <- c(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)

  for (i in 2:(d[1])) {
    x[,i+max.lag-1] <- r.mtx[i,(start.index-1-shift.indices[i-1]):(d[2]-1-shift.indices[i-1])]
  }

  end.training <- round(split[1] / 100 * d[2])

  end.validation <- round(sum(split[1:2]) / 100 * d[2])

  x <- x[,as.logical(c(rep(1, max.lag), mask))]

  y.train <- as.matrix(y[1:end.training], end.training, 1)
  x.train <- x[1:end.training,]

  y.valid <- as.matrix(y[(end.training+1):(end.validation)], end.validation-end.training, 1)
  x.valid <- x[(end.training+1):(end.validation),]

  y.test <- as.matrix(y[(end.validation+1):(d[2] - start.index + 1)],
d[2]-start.index-end.validation+1, 1)
  x.test <- x[(end.validation+1):(d[2] - start.index + 1),]

  list(x=x, y=y, x.train=x.train, y.train=y.train, x.valid=x.valid,
y.valid=y.valid, x.test=x.test, y.test=y.test)
}

```

The `log.lik.fcn()`, `mle.train()`, `vol.exp.sm()` and `norm.data()` functions are outlined below

```
log.lik.fcn <- function(lambda,x) {
  x<-as.numeric(x)
  n = length(x); sigma2 = x^2; loglik = 0
  for (t in 2:n){
    sigma2[t] = (1-lambda)*x[t-1]^2 + lambda*sigma2[t-1]
    loglik = loglik + (log(1/(2*pi*sigma2[t]))^0.5)+(-x[t]^2/(2*sigma2[t])))
  }
  return(-loglik)
}
```

```
mle.train <- function(x){
  mles <- numeric(ncol(x$x.train))
  for (i in 1:ncol(x$x.train)){
    v<-optim( par=.93, fn = log.lik.fcn, x = x$x.train[,i],
      method = "L-BFGS-B", lower = 0.0001,upper = 0.9999)
    mles[i]<-v$par
  }
  return(mles)
}
```

```
vol.exp.sm <- function(x, lambda) {
  # Exponential smoothing of x^2 with parameter lambda
  sigma2 <- x^2
  n <- length(x)
  for (i in 2:n)
    sigma2[i] <- sigma2[i-1] * lambda + x[i-1]^2 * (1-lambda)
  sigma <- sqrt(sigma2)
  resid <- x/sigma
  resid[is.na(resid)] <- 0
  sq.resid <- resid^2
  list(sigma2=sigma2, sigma=sigma, resid = resid, sq.resid = sq.resid)
}
```

#norm.data outputs a list with the normalised data to be used in calculating future returns

```
norm.data <- function(x){
  #x is the data returned by pred.r.prepare()
  #the function calls vol.exp.sm() (using mles as lambda)
  #to calculate daily volatilities and normalise our data

  mles<-mle.train(x)

  x.train.dev <- x$x.train
  y.train.dev <- x$y.train

  x.valid.dev <- x$x.valid
  y.valid.dev <- x$y.valid

  x.test.dev <- x$x.test
  y.test.dev <- x$y.test

  for (i in 1:ncol(x$x.train)) {
    v1 <- vol.exp.sm(x = x.train.dev[,i ], lambda = mles[i])
    x.train.dev[, i] <- v1$resid
  }
}
```

```

v2<- vol.exp.sm(x = x.valid.dev[,i ], lambda = mles[i])
x.valid.dev[, i] <- v2$resid
v3 <- vol.exp.sm(x = x.test.dev[,i], lambda = mles[i])
x.test.dev[, i] <- v3$resid
}

v <- vol.exp.sm(x$y.train, lambda= mle_y)
y.train.dev <- v$resid

v <- vol.exp.sm(x$y.valid, lambda= mle_y)
y.valid.dev <- v$resid

v <- vol.exp.sm(x$y.test,lambda= mle_y)
y.test.dev <- v$resid

list(y.train.dev=y.train.dev, x.train.dev=x.train.dev,
y.valid.dev=y.valid.dev, x.valid.dev=x.valid.dev,
y.test.dev=y.test.dev, x.test.dev=x.test.dev)
}

```

C Question 4 Code

```

thresh.reg <- function(x, y, th, x.pred = NULL) {

  # estimation of beta in y = a + x beta + epsilon (linear regression)
  # but only using those covariates in x whose marginal correlation
  # with y exceeds th
  # use th = 0 for full regression
  # note the intercept is added
  # x.pred is a new x for which we wish to make prediction

  d <- dim(x)
  ind <- (abs(cor(x, y)) > th)
  n <- sum(ind)

  new.x <- matrix(c(rep(1, d[1]), x[,ind]), d[1], n+1)    ## Adding intercept term

  gram = t(new.x) %*% new.x

  beta <- solve(gram) %*% t(new.x) %*% matrix(y, d[1], 1)

  ind.ex <- c(1, as.numeric(ind))

  ind.ex[ind.ex == 1] <- beta

  condnum = max(svd(gram)$d)/min(svd(gram)$d)

  pr <- 0

  if (!is.null(x.pred)) pr <- sum(ind.ex * c(1, x.pred))

  list(beta = ind.ex, pr=pr, condnum = condnum)
}

```

```

rolling.thresh.reg <- function(x, th, win, warmup, reg.function = thresh.reg) {

  # performs prediction over a rolling window of size win over the training set
  # x - returned by pred.footsie.prepare
  # th - threshold for thresh.reg
  # warmup - t_0 from the lecture notes

  xx <- norm.data(x)
  n <- length(xx$y.train.dev)
  err <- 0
  condnum <- predi <- truth <- rep(0, n-warmup+1)

  for (i in warmup:n) {
    y <- xx$y.train.dev[(i-win):(i-1)]
    xxx <- xx$x.train.dev[(i-win):(i-1),]
    zz <- reg.function(xxx, y, th, xx$x.train.dev[i,])
    predi[i-warmup+1] <- zz$pr
    condnum[i-warmup+1] <- zz$condnum
    truth[i-warmup+1] <- xx$y.train.dev[i]
  }
  #The returns below are changed to compensate for differing investment strategy

  i_s<-investment_strategy <- if_else (predi>0, 1, -1)
  ret <- i_s*truth

  err <- sqrt(250) * mean(ret) / sqrt(var(ret))
  list(err=err, predi=predi, truth=truth, condnum=condnum, ret=ret)
}

```

```

rolling.thresh.reg.valid <- function(x, th, win, warmup, reg.function = thresh.reg) {

  # The same as the previous function but for the validation set
  xx <- norm.data(x)
  n <- length(xx$y.valid.dev)
  err <- 0
  condnum <- predi <- truth <- rep(0, n-warmup+1)
  for (i in warmup:n) {
    y <- xx$y.valid.dev[(i-win):(i-1)]
    xxx <- xx$x.valid.dev[(i-win):(i-1),]
    zz <- reg.function(xxx, y, th, xx$x.valid.dev[i,])
    predi[i-warmup+1] <- zz$pr
    condnum[i-warmup+1] <- zz$condnum
    truth[i-warmup+1] <- xx$y.valid.dev[i]
  }
  i_s<-investment_strategy <- if_else (predi>0, 1, -1)
  ret <- i_s*truth
  err <- sqrt(250) * mean(ret) / sqrt(var(ret))
  list(err=err, predi=predi, truth=truth, condnum=condnum, ret=ret)
}

```

```

rolling.thresh.reg.test <- function(x, th, win, warmup, reg.function = thresh.reg) {
  # The same as the previous function but for the test set
  xx <- norm.data(x)
  n <- length(xx$y.test.dev)
  err <- 0
  condnum <- predi <- truth <- rep(0, n-warmup+1)
  for (i in warmup:n) {

```

```

y <- xx$y.test.dev[(i-win):(i-1)]
xxx <- xx$x.test.dev[(i-win):(i-1),]
zz <- reg.function(xxx, y, th, xx$x.test.dev[i,])
predi[i-warmup+1] <- zz$pr
condnum[i-warmup+1] <- zz$condnum
truth[i-warmup+1] <- xx$y.test.dev[i]
}
i_s<-investment_strategy <- if_else (predi>0, 1, -1)
ret <- i_s*truth
err <- sqrt(250) * mean(ret) / sqrt(var(ret))
list(err=err, predi=predi, truth=truth, condnum=condnum, ret=ret)
}

sharpe.curves <- function(x, th, warmup, reg.function = thresh.reg,
win = seq(from = 10, to = warmup, by = 10)) {
  # computes Sharpe ratios for a sequence of rolling windows (D in the lecture notes)
  # for the training, validation and test sets

  w <- length(win)

  train.curve <- valid.curve <- test.curve <- rep(0, w)

  n <- length(x$y.train)

  i=1
  rreg <- rolling.thresh.reg(x, th, win[i], warmup, reg.function)
  rreg.valid <- rolling.thresh.reg.valid(x, th, win[i], warmup, reg.function)
  rreg.test <- rolling.thresh.reg.test(x, th, win[i], warmup, reg.function)

  condnum = matrix(0,w,length(rreg$condnum))
  condnum.valid = matrix(0,w,length(rreg.valid$condnum))
  condnum.test = matrix(0,w,length(rreg.test$condnum))

  train.curve[i] <- rreg$err
  valid.curve[i] <- rreg.valid$err
  test.curve[i] <- rreg.test$err

  condnum[i,] <- rreg$condnum
  condnum.valid[i,] <- rreg.valid$condnum
  condnum.test[i,] <- rreg.test$condnum

  for (i in 2:w) {
    rreg <- rolling.thresh.reg(x, th, win[i], warmup, reg.function)
    rreg.valid <- rolling.thresh.reg.valid(x, th, win[i], warmup, reg.function)
    rreg.test <- rolling.thresh.reg.test(x, th, win[i], warmup, reg.function)

    train.curve[i] <- rreg$err
    valid.curve[i] <- rreg.valid$err
    test.curve[i] <- rreg.test$err

    condnum[i,] <- rreg$condnum
    condnum.valid[i,] <- rreg.valid$condnum
    condnum.test[i,] <- rreg.test$condnum
  }
  list(train.curve = train.curve, valid.curve = valid.curve, test.curve = test.curve, condnum=condnum,
condnum.valid = condnum.valid, condnum.test = condnum.test)

```

```
}
```

The output commented on in **Q4** was generated using the code below

```
q=0
data = pred.r.prepare(q)
sc0 = sharpe.curves(x=data, th=0, warmup= 250,
                    win = seq(from = 50, to = 250, by = 20))

sc0$train.curve
sc0$valid.curve
sc0$test.curve
vroll = rolling.thresh.reg.valid(x=data, 0, 250, 250)
plot.ts(vroll$predi, main='0 time lag prediction')
##Some positions are simply ridiculous!
##We will not invest like that in real world

q=1
data = pred.r.prepare(q)
data$x.train
sc1 = sharpe.curves(x=data, th=0, warmup= 250,
                    win = seq(from = 50, to = 250, by = 20))

sc1$train.curve
sc1$valid.curve
sc1$test.curve
vroll = rolling.thresh.reg.valid(x=data, 0, 250, 250)
plot.ts(vroll$predi, main='1 time lag prediction')
##Some positions are simply ridiculous!
##We will not invest like that in real world
```

D Q5 Code

```
fcctr.mdl.train <- function(factor, warmup, win, reg.function=thresh.reg) {
  x<- pred.r.prepare(max.lag=1)
  xx <- norm.data(x)
  n <- length(xx$y.train.dev)
  err <- 0

  condnum <- predi <- truth <- rep(0, n-warmup+1)

  for (i in warmup:n) {

    y <- xx$y.train.dev[(i-win):(i-1)] - mean(xx$y.train.dev[(i-win):(i-1)])
    c1 <- c(xx$x.train.dev[(i-win):(i-1),1] - mean(xx$x.train.dev[(i-win):(i-1),1]))
    c2<- c(xx$x.train.dev[(i-win):(i-1),2] - mean(xx$x.train.dev[(i-win):(i-1),2]))

    if (factor==1){
      xxx<-cbind(c1)
      train.pca <- prcomp(xxx)
      f2<-t(train.pca$rotation[,1])%*%t(train.pca$x)
      zz <- reg.function(t(f2), y, th=0, xx$x.train.dev[i]-mean(xx$x.train.dev[i-win:i]))
```

```

    predi[i-warmup+1] <- zz$pr
    condnum[i-warmup+1] <- zz$condnum
    truth[i-warmup+1] <- xx$y.train.dev[i]-mean(xx$y.train.dev[n])
  }

  if (factor==2){
    xxx<-cbind(c1,c2)
    train.pca <- prcomp(xxx)
    f2<-t(train.pca$rotation[,c(1,2)])%*%t(train.pca$x)
    zz <- reg.function(t(f2), y, th=0, xx$x.train.dev[i,]-mean(xx$x.train.dev[i-win:i,]))

    predi[i-warmup+1] <- zz$pr
    condnum[i-warmup+1] <- zz$condnum
    truth[i-warmup+1] <- xx$y.train.dev[i]-mean(xx$y.train.dev[n])
  }
}

i_s<-investment_strategy <- if_else (predi>0, 1, -1)
ret <- i_s*truth
err <- sqrt(250) * mean(ret) / sqrt(var(ret))

list(err=err, predi=predi, truth=truth, condnum=condnum, ret=ret)
}

fctr.mdl.valid <- function(factor, warmup, win,reg.function=thresh.reg) {
  #Same as last function but for validation set
  x<- pred.r.prepare(max.lag=1)
  xx <- norm.data(x)
  n <- length(xx$y.valid.dev)
  err <- 0
  condnum <- predi <- truth <- rep(0, n-warmup+1)
  for (i in warmup:n) {
    y <- xx$y.valid.dev[(i-win):(i-1)] - mean(xx$y.valid.dev[(i-win):(n-1)])
    c1 <- c(xx$x.valid.dev[(i-win):(i-1),1] - mean(xx$x.valid.dev[(i-win):(n-1),1]))
    c2<- c(xx$x.valid.dev[(i-win):(i-1),2] - mean(xx$x.valid.dev[(i-win):(n-1),2]))

    if (factor==1){
      xxx<-cbind(c1)
      valid.pca <- prcomp(xxx)
      f2<-t(valid.pca$rotation[,1])%*%t(valid.pca$x)
      zz <- reg.function(t(f2), y, th=0, xx$x.valid.dev[i]-mean(xx$x.valid.dev[i-win:i]))
      predi[i-warmup+1] <- zz$pr
      condnum[i-warmup+1] <- zz$condnum
      truth[i-warmup+1] <- xx$y.valid.dev[i]
    }
    if (factor==2){
      xxx<-cbind(c1,c2)
      valid.pca <- prcomp(xxx)
      f2<-t(valid.pca$rotation[,c(1,2)])%*%t(valid.pca$x)
      zz <- reg.function(t(f2), y, th=0, xx$x.valid.dev[i,]-mean(xx$x.valid.dev[i-win:i,]))
      predi[i-warmup+1] <- zz$pr
      condnum[i-warmup+1] <- zz$condnum
      truth[i-warmup+1] <- xx$y.valid.dev[i]
    }
  }
}

i_s<-investment_strategy <- if_else (predi>0, 1, -1)
ret <- i_s*truth
err <- sqrt(250) * mean(ret) / sqrt(var(ret))

```

```

    list(err=err, predi=predi, truth=truth, condnum=condnum, ret=ret)
  }

fctr.mdl.test <- function(factor, warmup, win, reg.function=thresh.reg) {
  #Same as last function but for test set
  x<- pred.r.prepare(max.lag=1)
  xx <- norm.data(x)
  n <- length(xx$y.test.dev)
  err <- 0
  condnum <- predi <- truth <- rep(0, n-warmup+1)
  for (i in warmup:n) {
    y <- xx$y.test.dev[(i-win):(i-1)] - mean(xx$y.test.dev[(i-win):(i-1)])
    c1 <- c(xx$x.test.dev[(i-win):(i-1),1] - mean(xx$x.test.dev[(i-win):(i-1),1]))
    c2<- c(xx$x.test.dev[(i-win):(i-1),2] - mean(xx$x.test.dev[(i-win):(i-1),2]))

    if (factor==1){
      xxx<-cbind(c1)
      test.pca <- prcomp(xxx)
      f2<-t(test.pca$rotation[,1])%*%t(test.pca$x)
      zz <- reg.function(t(f2), y, th=0, xx$x.test.dev[i]-mean(xx$x.test.dev[i-win:i]))

      predi[i-warmup+1] <- zz$pr
      condnum[i-warmup+1] <- zz$condnum
      truth[i-warmup+1] <- xx$y.test.dev[i]
    }
    if (factor==2){
      xxx<-cbind(c1,c2)
      test.pca <- prcomp(xxx)
      f2<-t(test.pca$rotation[,c(1,2)])%*%t(test.pca$x)
      zz <- reg.function(t(f2), y, th=0, xx$x.test.dev[i,]-mean(xx$x.test.dev[i-win:i,]))

      predi[i-warmup+1] <- zz$pr
      condnum[i-warmup+1] <- zz$condnum
      truth[i-warmup+1] <- xx$y.test.dev[i]
    }
  }
  i_s<-investment_strategy <- if_else (predi>0, 1, -1)
  ret <- i_s*truth
  err <- sqrt(250) * mean(ret) / sqrt(var(ret))
  list(err=err, predi=predi, truth=truth, condnum=condnum, ret=ret)
}

```

The function has been slightly altered from [sharpe.curves\(\)](#),

```

sharpe.curves.fctr <- function(factor, warmup, reg.function = thresh.reg,
win = seq(from = 10, to = warmup, by = 10)) {

  # computes Sharpe ratios for a sequence of rolling windows (D in the lecture notes)
  # for the training, validation and test sets

  w <- length(win)

  train.curve <- valid.curve <- test.curve <- rep(0, w)

```

```

i=1
rreg <- fctr.mdl.train(factor, warmup, win[i], reg.function)
rreg.valid <- fctr.mdl.valid(factor, warmup, win[i] , reg.function)
rreg.test <- fctr.mdl.test(factor, warmup, win[i] , reg.function)

condnum = matrix(0,w,length(rreg$condnum))
condnum.valid = matrix(0,w,length(rreg.valid$condnum))
condnum.test = matrix(0,w,length(rreg.test$condnum))

train.curve[i] <- rreg$err
valid.curve[i] <- rreg.valid$err
test.curve[i] <- rreg.test$err

condnum[i,] <- rreg$condnum
condnum.valid[i,] <- rreg.valid$condnum
condnum.test[i,] <- rreg.test$condnum

for (i in 2:w) {
  rreg <- fctr.mdl.train(factor, warmup, win[i], reg.function)
  rreg.valid <- fctr.mdl.valid(factor, warmup, win[i] , reg.function)
  rreg.test <- fctr.mdl.test(factor, warmup, win[i] , reg.function)

  train.curve[i] <- rreg$err
  valid.curve[i] <- rreg.valid$err
  test.curve[i] <- rreg.test$err

  condnum[i,] <- rreg$condnum
  condnum.valid[i,] <- rreg.valid$condnum
  condnum.test[i,] <- rreg.test$condnum
}

list(train.curve = train.curve, valid.curve = valid.curve, test.curve = test.curve,
condnum=condnum, condnum.valid = condnum.valid,
condnum.test = condnum.test)
}

```

The output commented on in **Q5** was generated using the code below

```

scf1<-sharpe.curves.fctr(factor=1,warmup=250,win = seq(from = 50, to = 250, by = 20))

scf1$train.curve
scf1$valid.curve
scf1$test.curve

scf2<-sharpe.curves.fctr(factor=2,warmup=250,win = seq(from = 50, to = 250, by = 20))

scf2$train.curve
scf2$valid.curve
scf2$test.curve

fctrroll1 = fctr.mdl.valid(factor=1,250, 250)
plot.ts(fctrroll1$predi,main='1 factor prediction')

fctrroll2 = fctr.mdl.valid(factor=2,250, 250)
plot.ts(fctrroll2$predi, main='2 factor prediction')

```
